

# Visor de Imágenes en Capas 3D

**Tecnologías:** Java 17+, Gradle, OpenGL 1.0 (JOGL), FlatLaf, LWJGL 2 (legacy) o JOGL

## Estructura de Carpetas

```
visor-capas-3d/
├── build.gradle
├── settings.gradle
├── README.md
├── docs/
│   ├── zbuffer-vs-wbuffer.md           # Explicación teórica
│   └── manual-usuario.md
└── src/main/java/com/team/visor3d/
    ├── Main.java
    ├── gui/
    │   ├── MainFrame.java
    │   ├── ControlPanel.java
    │   ├── LayerListPanel.java
    │   └── OpenGLPanel.java
    ├── model/
    │   ├── ImageLayer.java
    │   └── LayerManager.java
    ├── opengl/
    │   ├── GLRenderer.java
    │   └── TextureLoader.java
    └── util/
        └── ImageUtils.java
```

## Asignación de Tareas por Integrante

 **Dennis: Configuración del Proyecto + Estructura Base**

**Rama:** `feature/setup-proyecto`

**Dependencia:** Ninguna (primero en iniciar)

| Tarea | Descripción                                                                                                      |
|-------|------------------------------------------------------------------------------------------------------------------|
| 1.1   | Crear proyecto Gradle con <code>build.gradle</code> configurado                                                  |
| 1.2   | Añadir dependencias: JOGL (OpenGL 1.0), FlatLaf, JOML (math)                                                     |
| 1.3   | Crear estructura de paquetes ( <code>gui</code> , <code>model</code> , <code>opengl</code> , <code>util</code> ) |
| 1.4   | Crear clase <code>Main.java</code> con punto de entrada básico                                                   |
| 1.5   | Configurar Look and Feel FlatLaf en <code>Main</code>                                                            |

| Tarea | Descripción                                                      |
|-------|------------------------------------------------------------------|
| 1.6   | Crear <code>MainFrame.java</code> (JFrame base con BorderLayout) |

#### Archivos a entregar en PR:

- `build.gradle`, `settings.gradle`
- `src/main/java/com/team/visor3d/Main.java`
- `src/main/java/com/team/visor3d/gui/MainFrame.java`

#### Criterios de PR:

- ☐ Proyecto compila con `./gradlew build`
- ☐ Ventana principal se abre con estilo FlatLaf
- ☐ Estructura de paquetes creada y vacía lista para usar

---

### Diego: Modelo de Datos + Carga de Imágenes

Rama: `feature/modelo-carga-imagenes`

Dependencia: Integrante 1 (necesita estructura de paquetes)

| Tarea | Descripción                                                                                       |
|-------|---------------------------------------------------------------------------------------------------|
| 2.1   | Crear <code>ImageLayer.java</code> (POJO con: ID, ruta, textureID, posZ, colorRGB, alpha, nombre) |
| 2.2   | Crear <code>LayerManager.java</code> (lista observable de capas, métodos CRUD)                    |
| 2.3   | Implementar carga de imágenes desde disco (JFileChooser)                                          |
| 2.4   | Validar formatos soportados (PNG, JPG, BMP)                                                       |
| 2.5   | Generar thumbnails para vista previa                                                              |
| 2.6   | Crear <code>ImageUtils.java</code> (redimensionar, convertir a BufferedImage)                     |

#### Archivos a entregar en PR:

- `src/main/java/com/team/visor3d/model/ImageLayer.java`
- `src/main/java/com/team/visor3d/model/LayerManager.java`
- `src/main/java/com/team/visor3d/util/ImageUtils.java`

#### Criterios de PR:

- ☐ `LayerManager` puede agregar/eliminar capas
- ☐ JFileChooser filtra imágenes válidas
- ☐ Thumbnails se generan correctamente
- ☐ Tests unitarios para `ImageUtils` (redimensionado)

---

### Marielena: Panel de Control GUI + Lista de Capas

Rama: `feature/gui-contrroles`

Dependencia: Integrante 1 (MainFrame) + Integrante 2 (LayerManager)

| Tarea | Descripción                                                        |
|-------|--------------------------------------------------------------------|
| 3.1   | Crear <code>ControlPanel.java</code> (JPanel con controles)        |
| 3.2   | Botón "Cargar Imagen" (abre JFileChooser, notifica a LayerManager) |
| 3.3   | Selector de color (JColorChooser) para RGB                         |
| 3.4   | Slider de transparencia (JSlider 0-100 → Alpha 0.0-1.0)            |
| 3.5   | Slider de profundidad Z (JSlider -10.0 a 10.0)                     |
| 3.6   | Crear <code>LayerListPanel.java</code> (JTable con lista de capas) |
| 3.7   | Implementar selección de capa activa (click en tabla)              |
| 3.8   | Botón "Eliminar Capa"                                              |
| 3.9   | Sincronizar controles con capa seleccionada (listeners)            |

### Archivos a entregar en PR:

- `src/main/java/com/team/visor3d/gui/ControlPanel.java`
- `src/main/java/com/team/visor3d/gui/LayerListPanel.java`

### Criterios de PR:

- ☐ Controles actualizan el modelo `ImageLayer` seleccionado
- ☐ JTable muestra: Nombre, Z, Alpha, Color (preview)
- ☐ Al seleccionar capa, controles se actualizan con sus valores
- ☐ Cambios en controles disparan `repaint()` del panel OpenGL (stub)

## Mateo: Renderizado OpenGL 1.0 + Texturas

 \*\* Renderizado OpenGL 1.0 + Texturas\*\*

**Rama:** `feature/opengl-render`

**Dependencia:** Integrante 1 (MainFrame) + Integrante 2 (ImageLayer, LayerManager)

| Tarea | Descripción                                                                  |
|-------|------------------------------------------------------------------------------|
| 4.1   | Crear <code>OpenGLPanel.java</code> (extiende GLCanvas o GLJPanel)           |
| 4.2   | Crear <code>GLRenderer.java</code> (implementa GLEventListener)              |
| 4.3   | Configurar contexto OpenGL 1.0 (Fixed Pipeline)                              |
| 4.4   | Implementar <code>TextureLoader.java</code> (BufferedImage → textura OpenGL) |
| 4.5   | Renderizar planos (quads GL_QUADS) con texturas                              |
| 4.6   | Aplicar color RGB y alpha (glColor4f) sobre texturas                         |
| 4.7   | Implementar posicionamiento en eje Z (glTranslatef)                          |
| 4.8   | Configurar cámara básica (gluLookAt o glTranslatef en Z)                     |

| Tarea | Descripción                                        |
|-------|----------------------------------------------------|
| 4.9   | Habilitar <code>GL_DEPTH_TEST</code> para Z-Buffer |
| 4.10  | Implementar rotación básica de cámara (mouse drag) |

#### Archivos a entregar en PR:

- `src/main/java/com/team/visor3d/gui/OpenGLPanel.java`
- `src/main/java/com/team/visor3d/opengl/GLRenderer.java`
- `src/main/java/com/team/visor3d/opengl/TextureLoader.java`

#### Criterios de PR:

- ☐ Imágenes se renderizan como texturas en quads
- ☐ Color y alpha se aplican correctamente (`glColor4f * glTexEnv`)
- ☐ Capas en diferentes Z se visualizan con profundidad correcta
- ☐ `GL_DEPTH_TEST` funciona (objetos se ocultan correctamente)
- ☐ Interacción con mouse rota la vista

### Freddy: Integración Final

**Rama:** `feature/integracion-documentacion`

**Dependencia:** TODOS los anteriores

| Tarea | Descripción                                                                              |
|-------|------------------------------------------------------------------------------------------|
| 5.1   | Integrar todos los paneles en <code>MainFrame</code>                                     |
| 5.2   | Conectar <code>LayerManager</code> ↔ <code>ControlPanel</code> ↔ <code>GLRenderer</code> |
| 5.3   | Implementar notificación de cambios (Observer pattern)                                   |
| 5.4   | Añadir menú "Ayuda" con explicación de Z-Buffer vs W-Buffer                              |
| 5.5   | Crear documento <code>docs/zbuffer-vs-wbuffer.md</code>                                  |
| 5.6   | Crear <code>docs/manual-usuario.md</code>                                                |
| 5.7   | Añadir visualización de FPS                                                              |
| 5.8   | Resolver bugs de integración                                                             |
| 5.9   | Preparar demo para presentación                                                          |

#### Archivos a entregar en PR:

- `src/main/java/com/team/visor3d/gui/MainFrame.java` (actualizado)

#### Criterios de PR:

- ☐ Aplicación funciona end-to-end
- ☐ Documentación teórica explica Z-Buffer vs W-Buffer con ecuaciones
- ☐ No hay `NullPointerExceptions` en flujo normal

- ☐ Demo funciona: cargar 3+ imágenes, mover en Z, cambiar color/alpha

---

## Reglas de PR

1. **Cada integrante trabaja en su propia rama** `feature/nombre-tarea`
  2. **Antes de hacer PR**, hacer `git pull origin main` y resolver conflictos
  3. **PR debe incluir:**
    - Descripción de cambios
    - Checklist de criterios de aceptación
    - Screenshots (si aplica a GUI)
    - Mencionar al siguiente integrante que depende de esta PR
  4. **Revisor:** Otro integrante del equipo (rotativo)
  5. **Merge:** Solo después de 1 aprobación y CI passing (compilación)
- 

## Criterios de Aceptación por PR (Template)

```
## PR: [Nombre de la feature]

### Cambios realizados
- [ ] Lista de archivos modificados

### Cómo probar
1. `./gradlew build`
2. `./gradlew run`
3. Pasos específicos...

### Checklist
- [ ] Compila sin errores
- [ ] No hay warnings críticos
- [ ] Código comentado (JavaDoc)
- [ ] Dependencias del integrante anterior resueltas

### Screenshots
[Adjuntar si aplica]

### Siguiendo integrante
@IntegranteX - Esta PR desbloquea tu tarea de [descripción]
```

---

## build.gradle (Referencia para Integrante 1)

```
plugins {
    id 'java'
    id 'application'
}

group = 'com.team'
```

```
version = '1.0.0'

java {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}

repositories {
    mavenCentral()
}

dependencies {
    // FlatLaf - Look and Feel moderno
    implementation 'com.formdev:flatlaf:3.2.5'

    // JOGL - OpenGL 1.0 para Java
    implementation 'org.jogamp.jogl:joal-main:2.4.0'
    implementation 'org.jogamp.gluegen:gluegen-rt-main:2.4.0'

    // JOML - Matemáticas 3D (opcional pero útil)
    implementation 'org.joml:joml:1.10.5'

    // Test
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'
}

application {
    mainClass = 'com.team.visor3d.Main'
}

test {
    useJUnitPlatform()
}
```

---

## Consejos para el Equipo

1. **Comunicación diaria:** Usen un grupo de chat para avisar cuando un PR está listo
2. **Interfaces primero:** Definan las interfaces/clases públicas entre módulos al inicio
3. **Stubs/Mocks:** Si un integrante necesita algo del siguiente, crear stubs temporales
4. **Commits atómicos:** Un commit por funcionalidad, mensajes descriptivos en español
5. **Demo preparada:** Tengan 3-4 imágenes de prueba listas para la presentación